Compte rendu TP4 - MaintAI devient un package Python

Étudiante : Salma Riad
Groupe : à renseigner
Date : 10 novembre 2026
Module : Développement et déploiement d'applications intelligentes - séance 6

Objectif

Transformer MaintAI en package installable, créer la commande maintai-predict pour obtenir une prédiction sans API, puis construire un wheel utilisable dans un environnement distinct.

1. Mise en place

Le TP4 doit utiliser son propre environnement virtuel afin de ne pas modifier les dépendances des TP précédents :

```
python -m venv .venv
.\.venv\Scripts\activate.bat
pip install -r requirements.txt
python -m maintai.dataset
python -m maintai.train
python -m pytest
```

Sur le starter non installé, le polycopié prévoit **2 failed, 25 passed**. Les tests test_installed_version_matches_the_code et test_command_is_declared échouent car le package n'a pas encore été installé par pip et l'entrée console n'est pas enregistrée.

2. Sprint 1 - Configuration et installation éditable

Les éléments ajoutés à pyproject.toml :

```
[project]
name = "maintai"
dynamic = ["version"]
description = "Prédiction de panne à partir de relevés de capteurs industriels"
requires-python = ">=3.11"
dependencies = [
    "numpy>=2.0",
    "pandas>=2.2",
    "scikit-learn>=1.5",
    "joblib>=1.4",
    "httpx>=0.27",
]

[project.scripts]
maintai-predict = "maintai.cli:main"
```

Les dépendances obligatoires permettent d'utiliser les modules fondamentaux de MaintAI. pytest appartient au groupe optionnel dev, fastapi au groupe api et streamlit au groupe ui. Elles ne sont pas nécessaires pour exécuter seulement la commande de prédiction.

Installation et vérification :

```
pip install -e .
pip show maintai
maintai-predict 72 6 85 15000
python -m pytest
```

Extraits réellement observés :

```
Name: maintai
Version: 0.2.0
Summary: Prédiction de panne à partir de relevés de capteurs industriels

Probabilité de panne : 95 % (ALERTE)

.........................                                                [100%]
27 passed
```

L'option -e installe un lien vers le dossier source. Même depuis un autre répertoire, maintai-predict et import maintai utilisent les sources de développement ; maintai.__file__ pointe vers maintai/__init__.py du projet. Modifier le texte imprimé dans cli.py serait immédiatement visible après une nouvelle exécution, sans réinstallation. En revanche, changer la déclaration [project.scripts] nécessite de réinstaller les métadonnées.

3. Sprint 2 - Construction et livraison du wheel

Commande de construction exécutée :

```
python -m build
```

Fichiers générés dans dist/ :

```
maintai-0.2.0.tar.gz
maintai-0.2.0-py3-none-any.whl
```

L'inspection du wheel a montré les fichiers maintai/__init__.py, cli.py, client.py, config.py, data.py, dataset.py, predict.py, train.py, les modules du sous-package maintai/api/ ainsi que les métadonnées maintai-0.2.0.dist-info. Le wheel ne contient **ni le modèle (`models/`), ni les tests, ni `ui/`** : la configuration de découverte n'intègre que le package maintai.

Essai du wheel dans un environnement neuf

Un autre environnement virtuel a été créé pour installer le fichier wheel avec les bibliothèques déjà disponibles. La commande de prédiction a été lancée sans puis avec le chemin vers les artefacts entraînés.

Essai	Résultat observé
Sans MAINTAI_MODELS_DIR	Modèle introuvable : le wheel ne fournit pas models/model.joblib.
Avec MAINTAI_MODELS_DIR pointant vers les modèles du TP4	Probabilité de panne : 95 % (ALERTE)
En installant scikit-learn 1.8.0	La même version ayant servi à l'entraînement est déjà présente : aucun avertissement de compatibilité n'est attendu.

Sur Windows, le protocole à reproduire est :

```
python -m venv .venv
.\.venv\Scripts\activate.bat
pip install C:\CHEMIN\TP4\dist\maintai-0.2.0-py3-none-any.whl
maintai-predict 72 6 85 15000
set MAINTAI_MODELS_DIR=C:\CHEMIN\TP4\models
maintai-predict 72 6 85 15000
```

Le champ "sklearn_version" de models/model_info.json, enregistré dans maintai/train.py, indique la version qui a créé le fichier model.joblib. Si une autre version est utilisée lors du chargement, InconsistentVersionWarning peut apparaître. Lors de la vérification, la version d'entraînement était **1.8.0**, comme dans requirements.txt.

Le package à livrer nécessite donc trois éléments : le **code** (wheel), les **artefacts de modèle** (model.joblib et model_info.json) et les **versions de dépendances** compatibles. metrics.json permet également de conserver les mesures de l'entraînement.

4. Questions de compréhension

1. **`requirements.txt` contre `dependencies` ?** requirements.txt fixe les versions exactes (==) pour reproduire l'entraînement ; pyproject.toml indique les versions minimales (>=) pour permettre l'installation à côté d'autres packages.
2. **`pip install -e .` contre wheel ?** L'installation éditable pointe vers les sources : elle convient au développement. Le wheel installe une copie construite : il convient à la livraison et ne se met pas à jour lorsque les sources changent.
3. **Qu'est-ce qu'un point d'entrée ?** [project.scripts] associe maintai-predict à la fonction main de maintai/cli.py. Sur Windows, le script est normalement créé dans .venv\Scripts\maintai-predict.exe.

5. Livraison Git

- .gitignore exclut .venv/, dist/, build/, *.egg-info/, data/*.csv, models/ et .env.
- **Pull request TP4 :** lien à compléter après publication sur GitHub.
- **Tag `tp4` :** à créer après fusion.
- **`git status` avant commit :** à vérifier dans le dépôt du groupe ; l'archive d'origine ne contient pas l'historique Git.

6. Bilan

Le package **MaintAI 0.2.0** est installable en mode éditable ; sa commande renvoie **95 % (ALERTE)** sur le relevé d'exemple, les **27 tests automatisés passent**, et le wheel est construit et inspecté. La démonstration finale sur Windows et les opérations Git du groupe restent à confirmer.